XEBIA Day 4 (Individual)

TASK 1

```
#Q1
arr = list(map(int, input().split()))
first = second = float('-inf')

for i in arr:
    if i > first:
        second = first
        first = i
    elif i != first and i > second:
        second = i

print(second if second != float('-inf') else -1)

#Q2
arr = list(map(int, input().split()))
k = int(input())

n = len(arr)
k = k % n

def reverse(l, r):
    while l < r:
        arr[l], arr[r] = arr[r], arr[l]
        l += 1
        r -= 1

reverse(0, n - 1)
reverse(0, k - 1)
reverse(k, n - 1)
print(arr)

#Q3
s1 = input().strip()
s2 = input().strip()

s1 = s1.replace(" ", "").lower()
s2 = s2.replace(" ", "").lower()

if len(s1) != len(s2):
    print("false")
else:
    freq = {}
    for i in s1:
        freq[i] = freq.get(i, 0) + 1
    for i in s2:
        if i not in freq:
```

```
            print("false")
            break
        freq[i] -= 1
        if freq[i] < 0:
            print("false")
            break
    else:
        print("true")

#Q4
s = input().strip()

my_set = set()
l = 0
max_len = 0

for r in range(len(s)):
    while s[r] in my_set:
        my_set.remove(s[l])
        l += 1
    my_set.add(s[r])
    max_len = max(max_len, r - l + 1)

print(max_len)

-- Q5
SELECT d.dept_name, e.name, e.salary
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
JOIN (
    SELECT dept_id, MAX(salary) AS max_salary
    FROM Employees
    GROUP BY dept_id
) m ON e.dept_id = m.dept_id AND e.salary = m.max_salary;

-- Q6
SELECT
    DATE_FORMAT(order_date, '%b') AS month,
    SUM(amount) AS total_revenue,
    COUNT(order_id) AS total_orders
FROM Orders
WHERE YEAR(order_date) = 2024
GROUP BY MONTH(order_date), DATE_FORMAT(order_date, '%b')
ORDER BY total_revenue DESC;

#Q7
import requests

url = "https://jsonplaceholder.typicode.com/posts?userId=3"
```

```
try:
    res = requests.get(url, timeout=5)
    res.raise_for_status()
    data = res.json()

    for post in data:
        print(post["title"].upper())

    print(f"Total posts: {len(data)}")

except requests.exceptions.RequestException as e:
    print(f"Error fetching data: {e}")

#Q8
import requests

city = "Chandigarh"
lat = 30.6970
lon = 76.7491

url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true"

weather_map = {
    0: "Clear sky",
    1: "Mainly clear",
    2: "Partly cloudy",
    3: "Overcast",
    45: "Fog",
    61: "Light rain",
    80: "Rain showers",
    95: "Thunderstorm"
}

try:
    res = requests.get(url, timeout=5)
    res.raise_for_status()
    data = res.json()["current_weather"]

    temp = data["temperature"]
    code = data["weathercode"]
    condition = weather_map.get(code, "Unknown")

    print(f"City       : {city}")
    print(f"Temperature: {temp}°C")
    print(f"Condition  : {condition}")

except requests.exceptions.RequestException as e:
    print(f"Error fetching weather data: {e}")
```

TASK 2 - Check on Github

https://github.com/agam1092005/Xebia_day4/tree/main/task2

TASK 3

```
#BUG1
import requests

API_BASE_URL = "https://hr-internal.company.com/api"
API_TOKEN = "your_api_token_here"

def get_employee(employee_id):
    url = f"{API_BASE_URL}/employees/{employee_id}"

    # Added Authorization header with Bearer token to fix 401 Unauthorized
    response = requests.get(url, headers={"Authorization": f"Bearer {API_TOKEN}"})

    if response.status_code == 404:
        return None

    data = response.json()

    # Safely access "employee" key to avoid KeyError when response is 404
    return data.get("employee")

#BUG2
def get_monthly_attendance(conn, employee_id, month, year):
    # Changed LEFT JOIN to INNER JOIN to ensure only matching employee records are returned
    # Added employee_id filter to restrict results to the given employee
    query = """
        SELECT a.date, a.check_in, a.check_out, e.name
        FROM attendance a
        INNER JOIN employees e
        ON a.employee_id = e.id
        WHERE a.employee_id = ?
        AND a.month = ?
        AND a.year = ?
        ORDER BY a.date ASC
    """
    cursor = conn.cursor()
    cursor.execute(query, (employee_id, month, year))
    return cursor.fetchall()

#BUG3
from datetime import datetime
```

```
WORK_START = "09:00"
LATE_THRESHOLD_MINUTES = 15

def calculate_hours(check_in: str, check_out: str):
    fmt = "%H:%M"
    ci = datetime.strptime(check_in, fmt)
    co = datetime.strptime(check_out, fmt)

    # Corrected subtraction order to compute positive work duration (check_out - check_in)
    duration = co - ci
    hours_worked = duration.seconds / 3600

    start = datetime.strptime(WORK_START, fmt)
    late_by = (ci - start).seconds // 60

    # Changed condition to >= so exactly 15 minutes late is also flagged
    is_late = late_by >= LATE_THRESHOLD_MINUTES

    return round(hours_worked, 2), is_late

#BUG4
def generate_summary(records):
    total_hours = 0
    late_days = 0

    # Fixed loop range to prevent IndexError (removed +1)
    for i in range(len(records)):
        record = records[i]
        total_hours += record["hours"]
        if record["is_late"]:
            late_days += 1

    # Handled empty list to avoid division by zero when calculating average
    avg_hours = total_hours / len(records) if len(records) > 0 else 0

    return {
        "total_hours": round(total_hours, 2),
        "avg_hours": round(avg_hours, 2),
        "late_days": late_days,
        "days_present": len(records)
    }

MIN_DAYS_REQUIRED = 20
MAX_LATE_DAYS = 3

#BUG5
def check_attendance_policy(summary):
    days_present = summary["days_present"]
    late_days = summary["late_days"]
```

```
# Corrected comparison to flag employees with fewer than minimum required days
below_minimum = days_present < MIN_DAYS_REQUIRED

# Changed condition to >= so exactly max late days also triggers warning
exceeded_late = late_days >= MAX_LATE_DAYS

if below_minimum or exceeded_late:
    return {
        "warning": True,
        "reason": []
          + (["Below minimum attendance"] if below_minimum else [])
          + (["Exceeded late check-ins"] if exceeded_late else [])
    }
return {"warning": False, "reason": []}
```